

Document number: P2037r1
Date: 2020-06-15
Audience: LEWG
Reply-to: Andrzej Krzemiński <akrzemi1 at gmail dot com>

String's gratuitous assignment

This paper explores the capability of the assignment from `char` to `std::string` and the consequences of removing it. We propose to deprecate this assignment, not necessarily with the intention to remove it in the future.

Revision history

R0 → R1

1. Showing `getchar()` as a more realistic example of a correct use case of using `int` as `char`.
2. Added wording for the deprecation of the assignment, per LEWG recommendation.
3. Highlighted in discussion that there is no estimation of how many users will be negatively impacted by the deprecation.
4. Added observation that a simple removal of the assignment from `char` will automatically enable the the assignment from literal `0` which is UB.

Background

The interface of `std::basic_string` provides the following signature:

```
constexpr basic_string& operator=(charT c);
```

This allows the direct assignment from `char` to `std::string`:

```
std::string s;  
s = 'A';  
assert(s == "A");
```

However, due to the implicit conversion between scalar types, this allows an assignment from numeric types, such as `int` or `double`, which often has an undesired semantics:

```
std::string s;  
s = 50;  
assert(s == "2");  
  
s = 48.0;  
assert(s == "0");
```

In fact, any user-defined type that has an implicit conversion operator to `int` or `double` is also assignable to `std::string`.

In order to prevent the likely inadvertent conversions, [\[RU013\]](#) proposes to change the signature so that it is equivalent to:

```
template <class T>  
    requires is_same_v<T, charT>  
constexpr basic_string& operator=(charT c);
```

Discussion

Intended usage

Even the intended usage of the assignment from `char` is suspicious. We have a direct interface for assigning a single character to an existing `std::string`:

```
std::string s;  
s = 'A';
```

However, there is no corresponding interface — in the form of constructor — for initializing a string from a single character. We have to use a more verbose syntax:

```
const std::string s1(1u, 'C');  
const std::string s2 = {'C'};
```

Whatever the motivation for the assignment from `char` was, surely the same motivation applied for the converting constructor.

Common pitfall

There are two common situations where the gratuitous converting assignment from `int` to `std::string` is used inadvertently and results in a well-formed C++ program that does something else than what the programmer intended.

First is when inexperienced C++ programmers try to use their experience from weakly typed languages when trying to convert from `int` to `std::string` through an assignment syntax:

```
template <typename From, typename To>
    requires std::is_assignable_v<To&, From const&>
void convert(From const& from, To& to)
{
    to = from;
}

std::string s;
convert(50, s);
std::cout << s; // outputs "2"
```

The second situation is when a piece of data used throughout a program, such as a unique identifier, is changed type from `int` to `std::string`. Consider the common concept of an "id". While the concept is common and universally understood, there exists no natural internal representation of an identifier. It can be represented by an `int` or by a `std::string`, and sometimes the representation can change in time. If we decide to change the representation in our program, the expectation is that after the change whenever a raw `int` is converted to an id — either in initialization or in the assignment — a compiler should detect a type mismatch and report a compile-time error. But because of the surprising "conversion" this is not the case.

Valid conversions from int

There are usages of the assignment from type `int` to `std::string` that are nonetheless valid and behave exactly as intended. These are the cases when we already treat the value stored in an `int` as a character, but we store it in a variable of type `int` either for convenience or because of the peculiar rules of type promotions in C++. The first case is when we use literal `0` to indicate a null character `'\0'`:

```
if (auto ch = std::getchar(); ch != EOF) { // "Almost Always Auto" philosophy
    str = ch;
}
```

Function `std::getchar()` returns `int` so that, apart from any `char` value, it can also return special value `EOF`. But once we have confirmed the return value is not `EOF` we can treat the value as `char`.

Sometimes we may not even be aware that we are producing a value of type `int`:

```
void assign_digit(int d, std::string& s)
// precondition: 0 <= d && d <= 9
{
    constexpr char zero = '0';
    s = (char)d + zero;
}
```

In the example above we might believe that because we are adding two `chars`, the resulting type will also be of type `char`, but the result of the addition of two `chars` is in fact of type `int`. This incorrect expectation is enforced by the way narrowing is defined in C++:

```
// test if char + char == char :
constexpr char zero = '0';
const int d = 9;
char ch {(char)d + zero}; // brace-init prevents narrowing
```

Brace initialization prevents narrowing. The above "test" compiles fine, so no narrowing occurs. From this, a programmer could draw an incorrect conclusion that the type of expression `(char)d + zero` must be `char`; but it is not.

Our options

There is a number of ways we can respond to this problem.

Do nothing

That is, do not modify the interface of `std::basic_string`. The potential bugs resulting from the suspicious conversion can be detected by static analyzers rather than compilers. For instance, clang-tidy has checker [bugprone-string-integer-assignment](#) that reports all places

where the suspicious assignment from an `int` is performed. This avoids any correct code breakage, and leaves the option for the bugs to be detected by other tools.

Remove the assignment operator from `charT`

We can just remove the assignment from `charT` altogether. This assignment is suspicious even if no conversions are applied. It is like an assignment of a container element to a container. This warrants the usage of syntax that expresses the element-container relation, like:

```
str.assign(1, ch);
str = {ch};
```

A migration procedure can be provided for changing the program that previously used the suspicious assignment.

However, it should be noted that currently owing to the existence of the assignment from `char` the following code fails to compile:

```
str = 0;
str = NULL;
```

This is because there are two competing assignment operators: one taking `char` and the other taking `const char *`. If we removed the former assignment, the latter would start compiling, but the assignment from a null `const char *` would cause Undefined Behavior. In order to avoid current bugs and not introduce the potential for new ones, the removal of one assignment operator would have to be accompanied by the addition of another:

```
constexpr basic_string& operator=(nullptr_t) = delete;
```

An alternative solution would be to declare the assignment from `char` itself as deleted.

Deprecate the assignment

A softer variant of the above would be to declare the assignment from `charT` as deprecated. This does not affect the semantics of any existing program, and at the same time encourages tools (compilers included) to diagnose any usage of such assignment.

A deprecation is *not* a commitment to remove a feature ever in the future. A possible outcome of such deprecation would be that we will keep the assignment forever. Nonetheless, it should be noted that if the deprecated assignment is ever removed, it would introduce the problem of reenabling assignment from literal 0.

Poison the conversion from scalar types to `charT` in the assignment

Do what [\[RU013\]](#) proposes: replace the current signature of the assignment with something equivalent to:

```
template <class T>
requires is_same_v<T, charT>
constexpr basic_string& operator=(charT c);
```

This may still compromise some valid programs, but the damage is smaller than if the operator was removed altogether. An automated mechanical fix can be easily provided: you just need to apply a cast:

```
str = std::char_traits<char>::to_char_type(i);
```

This solution also suffers from the problem of reenabling assignment from literal 0.

Poison all conversion but the one from `int`

There is no controversy about disallowing an assignment from `float` or `unsigned int`. Chances that such usages are correct are so small that sacrificing them would be acceptable. The only assignment from non-`charT` that could be potentially correct is the one from `int`, as `ints` are often produced from `char` in unexpected places. Given that, we could poison other assignments, but leave the assignment from `int` intact.

However, all places where this bug has been reported, it was exactly the assignment from `int`, so this option may not be much more attractive than doing nothing.

Offer an alternative interface

If the assignment is narrowed in applicability or removed, this change can be accompanied by adding a dedicated interface for putting a single character into a string. we could add the following signature to `basic_string`:

```
constexpr basic_string& assign_char(charT c);
```

And this avoids any pitfalls, even if an `int` is passed to it:

```
str.assign_char('0' + 0); // we obviously mean a numeric conversion to char
```

It is superior to `str = {ch}` because it allows correct assignments from `int`, and it is superior to `str = {char(ch)}` because it avoids explicit conversion operators.

Impact on users

There was a consensus in LEWG to deprecate the assignment from `CharT` by moving it to Annex D. So we are only discussing the impact of deprecating the assignment. Deprecation technically does not alter the interface in the sense that programs that used to be valid remain valid with unaltered semantics, and programs that used to be invalid remain invalid with the same diagnostics. However, deprecation will impact the users who configure their compilers to warn about the usage of deprecated features and to treat warnings as errors. For users who use the string assignment inadvertently and incorrectly this breakage will be a gain. But for users who are aware of the semantics and assign from `int` to `string` consciously this will be a harm. The `int`-to-`string` assignment can be treated as a dangerous but useful tool. Such impact could be mitigated if compilers allow the users to control which deprecations are warned about.

The deprecation warning about the `int`-to-`string` assignment has not been implemented on any compiler that we are aware of. (It is implemented in clang-tidy though.) The impact on the users has not been estimated.

Proposed wording

Changes are relative to [\[N4861\]](#).

In `[basic.string]` paragraph 3, remove the the declaration of the assignment from `CharT` from class synopsis:

```
// 21.3.2.2, construct/copy/destroy
constexpr basic_string() noexcept(noexcept(Allocator())) : basic_string(Allocator()) { }
constexpr explicit basic_string(const Allocator& a) noexcept;
constexpr basic_string(const basic_string& str);
constexpr basic_string(basic_string&& str) noexcept;
constexpr basic_string(const basic_string& str, size_type pos,
                        const Allocator& a = Allocator());
constexpr basic_string(const basic_string& str, size_type pos, size_type n,
                        const Allocator& a = Allocator());
template<class T>
    constexpr basic_string(const T& t, size_type pos, size_type n,
                           const Allocator& a = Allocator());
template<class T>
    constexpr explicit basic_string(const T& t, const Allocator& a = Allocator());
constexpr basic_string(const charT* s, size_type n, const Allocator& a = Allocator());
constexpr basic_string(const charT* s, const Allocator& a = Allocator());
constexpr basic_string(size_type n, charT c, const Allocator& a = Allocator());
template<class InputIterator>
    constexpr basic_string(InputIterator begin, InputIterator end,
                           const Allocator& a = Allocator());
constexpr basic_string(initializer_list<charT>, const Allocator& = Allocator());
constexpr basic_string(const basic_string&, const Allocator&);
constexpr basic_string(basic_string&&, const Allocator&);
constexpr ~basic_string();
constexpr basic_string& operator=(const basic_string& str);
constexpr basic_string& operator=(basic_string&& str)
    noexcept(allocator_traits<Allocator>::propagate_on_container_move_assignment::value ||
             allocator_traits<Allocator>::is_always_equal::value);
template<class T>
    constexpr basic_string& operator=(const T& t);
constexpr basic_string& operator=(const charT* s);
constexpr basic_string& operator=(charT c);
constexpr basic_string& operator=(initializer_list<charT>);
```

Remove paragraph 30 from `[string.cons]`:

```
constexpr basic_string& operator=(const charT* s);

Effects: Equivalent to: return *this = basic_string_view<charT, traits>(s);

constexpr basic_string& operator=(charT c);

Effects: Equivalent to:
```

```
return *this = basic_string_view<charT, traits>(addressof(c), 1);
```

```
constexpr basic_string& operator=(initializer_list<charT> il);
```

Effects: Equivalent to:

```
return *this = basic_string_view<charT, traits>(il.begin(), il.size());
```

Modify section D.19 as follows (this includes changing the stable links):

D.19 Deprectaed basic_string capacity members

[depr.string.capacity]

The following members are is declared in addition to those members specified in 21.3.2.2 and 21.3.2.4:

```
namespace std {  
    template<class charT, class traits = char_traits<charT>,  
            class Allocator = allocator<charT>>  
    class basic_string {  
    public:  
        constexpr basic_string& operator=(charT c);  
        void reserve();  
    };  
}
```

```
constexpr basic_string& operator=(charT c);
```

Effects: Equivalent to:

```
return *this = basic_string_view<charT, traits>(addressof(c), 1);
```

```
void reserve();
```

Effects: After this call, `capacity()` has an unspecified value greater than or equal to `size()`. [Note: This is a non-binding shrink to fit request. —end note]

Acknowledgements

I am grateful to Antony Polukhin and Jorg Brown for their useful feedback. I am also grateful to Tomasz Kamiński for reviewing the proposed wording. Barry Revzin and Ville Voutilainen stressed the importance of estimating the impact of the deprecation on the users. This is now reflected in the paper.

References

1. [RU013] -- [string.cons].30
(<https://github.com/cplusplus/nbballot/issues/13>).
2. [CLANG] -- "Extra Clang Tools 10 documentation"
(<https://clang.llvm.org/extra/clang-tidy/3>).